

Patchline

Complete source code

Built by Rishik Rontala · 6 October 2026

Public repository: github.com/rishikrrontala-bot/patchline

This archive includes every source file for the browser prototype, tests, demo recorder, and PDF generators. Generated media and PDFs are excluded.

- 01 `index.html`
- 02 `demo.html`
- 03 `styles.css`
- 04 `app.js`
- 05 `engine.js`
- 06 `tests/engine.test.mjs`
- 07 `submission/make_onepager.py`
- 08 `submission/record-demo.cjs`
- 09 `submission/make_code_pdf.py`

All wound readings and labels in the prototype are synthetic.

The code cannot diagnose infection or operate treatment hardware.

index.html

```
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1">
6    <meta name="theme-color" content="#f4f1ea">
7    <meta name="author" content="Rishik Rontala">
8    <meta name="description" content="Patchline is an interactive, synthetic digital twin of a
9  > proposed smart bandage. Simulation only; no medical use.">
10   <title>Patchline — The signal between visits</title>
11   <link rel="stylesheet" href="./styles.css">
12   <script type="module" src="./app.js"></script>
13 </head>
14 <body>
15   <a class="skip-link" href="#main">Skip to prototype</a>
16   <div class="page-shell">
17     <header class="masthead" aria-label="Site header">
18       <a class="brand" href="#top" aria-label="Patchline home"><span class="brand-mark"
19 > aria-hidden="true"><i></i></i></i></i></i></span><span>PATCHLINE<span
20 > class="brand-dot">.</span></span></a>
21       <div class="masthead-center">RESEARCH PROTOTYPE <span aria-hidden="true"></span>
22 > 2026</div>
23       <div class="masthead-status"><span class="status-dot" aria-hidden="true"></span>
24 > SIMULATION ONLY</div>
25     </header>
26
27     <main id="main">
28       <section id="top" class="hero" aria-labelledby="hero-title">
29         <div class="hero-copy">
30           <div class="section-index"><span class="index-bar"></span> 00 / A WOUND-SENSING
31 > DIGITAL TWIN</div>
32           <h1 id="hero-title">The signal<br><em>between</em> visits<span
33 > class="period">.</span></h1>
34           <p class="hero-lede">A proposed bandage that watches four wound-environment signals
35 > and shows how a bounded orchestrator could respond. Explore the entire loop with synthetic
36 > data.</p>
37           <div class="hero-action-row">
38             <a class="primary-link" href="#prototype">Explore prototype <span
39 > aria-hidden="true">□</span></a>
40             <span class="hero-fineprint">No sensors connected. No treatment delivered.</span>
41           </div>
42           <div class="patch-panel" role="img" aria-label="Diagram of a conceptual smart bandage
43 > with four virtual sensor channels">
44             <div class="patch-panel-head"><span>PL-01 / CONCEPT PATCH</span><span>FIG.
45 > A</span></div>
46             <div class="patch-visual">
47               <div class="patch-label patch-label-t"><span
48 > class="callout-line"></span><span>01&nbsp;&nbsp;&nbsp; TEMPERATURE</span></div>
49               <div class="patch-label patch-label-m"><span
50 > class="callout-line"></span><span>02&nbsp;&nbsp;&nbsp; MOISTURE</span></div>
51               <div class="patch-label patch-label-p"><span
52 > class="callout-line"></span><span>03&nbsp;&nbsp;&nbsp; FLUID pH</span></div>
53               <div class="patch-label patch-label-g"><span
54 > class="callout-line"></span><span>04&nbsp;&nbsp;&nbsp; FLUID GLUCOSE*</span></div>
55               <div class="patch-shape">
56                 <div class="patch-wing patch-wing-left"></div>
57                 <div class="patch-core">
58                   <div class="patch-core-grid"></div>
59                   <div class="patch-center">
60                     <div class="patch-ring patch-ring-a"></div><div class="patch-ring
61 > patch-ring-b"></div>
```

index.html / continued

```

46         <div class="patch-cross">+</div>
47         <span class="node node-t"></span><span class="node node-m"></span><span
> class="node node-p"></span><span class="node node-g"></span>
48     </div>
49 </div>
50     <div class="patch-wing patch-wing-right"></div>
51 </div>
52 </div>
53     <div class="patch-panel-foot"><span>VIRTUAL BOARD / SENSOR
> LAYOUT</span><span>*EXPERIMENTAL, ARBITRARY UNITS</span></div>
54 </div>
55 </section>
56
57     <div class="safety-band"><span class="safety-sign">!</span><span><strong>Proof of system
> design.</strong> Every reading, score, and proposed actuation state below is simulated.
> Patchline cannot diagnose infection or choose care.</span><a href="#boundaries">View boundaries
> </a></div>
58
59     <section id="prototype" class="prototype-section" aria-labelledby="prototype-title">
60         <div class="section-heading">
61             <div><div class="section-index"><span class="index-bar"></span> 01 / INTERACTIVE
> PROTOTYPE</div><h2 id="prototype-title">A scenario, <em>all the way through.</em></h2></div>
62             <p>Choose a synthetic trace. Adjust the last sample to see the quality gate and
> simulated decision update.</p>
63         </div>
64
65         <div class="scenario-bar" role="group" aria-label="Synthetic scenario">
66             <div class="scenario-label">SELECT A TRACE <span>□</span></div>
67             <div class="scenario-controls">
68                 <button type="button" class="scenario-button" data-scenario="stable"
> aria-pressed="true"><span class="scenario-number">01</span><span>Stable</span><span
> class="scenario-arrow" aria-hidden="true">□</span></button>
69                 <button type="button" class="scenario-button" data-scenario="change"
> aria-pressed="false"><span class="scenario-number">02</span><span>Pattern change</span><span
> class="scenario-arrow" aria-hidden="true">□</span></button>
70                 <button type="button" class="scenario-button" data-scenario="fault"
> aria-pressed="false"><span class="scenario-number">03</span><span>Sensor fault</span><span
> class="scenario-arrow" aria-hidden="true">□</span></button>
71             </div>
72         </div>
73
74         <div class="workbench">
75             <div class="trace-panel">
76                 <div class="module-header"><div><span class="module-id">PL /
> TELEMETRY</span><h3>Four-channel signal</h3></div><div class="module-header-right"><span
> id="sample-count">— samples</span><span class="live-tag"><span></span></span>
> SYNTHETIC</span></div></div>
77                 <div id="trace-grid" class="trace-grid" aria-live="polite"></div>
78                 <p class="module-caption">* Wound-fluid glucose is an experimental signal shown in
> arbitrary units. It does not represent blood glucose. Traces use generated data, not patient
> measurements.</p>
79             </div>
80             <aside class="decision-panel" aria-labelledby="decision-title">
81                 <div class="module-header module-header-dark"><div><span class="module-id">PL /
> ORCHESTRATOR</span><h3 id="decision-title">Control plane</h3></div><span
> class="decision-step">02 / 03</span></div>
82                 <div class="quality-row"><span>DATA QUALITY</span><strong
> id="quality-value">—</strong></div>
83                 <div class="score-block"><div class="score-label">SIMULATED PATTERN SCORE <button
> type="button" class="info-button" id="score-info" aria-label="What does simulated pattern score
> mean?" title="A score trained on synthetic traces; not an infection
> probability.">?</button></div><div class="score-value"><span id="score-value">—</span><span>

```

index.html / continued

```

> class="score-denominator">/100</span></div><p id="score-caption">Trained only on generated
> traces. Not an infection probability.</p></div>
84     <div class="decision-outcome"><span class="decision-eyebrow">BOUNDED
> DECISION</span><h4 id="decision-label">Loading scenario...</h4><p id="decision-rationale">The
> quality gate runs before the simulated orchestrator.</p></div>
85     <div class="virtual-outputs"><div class="virtual-outputs-head"><span>HYPOTHETICAL
> OUTPUTS</span><span>ABSTRACT 0-3 UNITS</span></div><div
> class="output-line"><span>Ultrasound</span><div class="level-track" id="ultrasound-track"
> aria-hidden="true"><i></i><i></i><i></i></div><strong id="ultrasound-value">0 /
> 3</strong></div><div class="output-line"><span>Violet light</span><div class="level-track"
> id="violet-track" aria-hidden="true"><i></i><i></i><i></i></div><strong id="violet-value">0 /
> 3</strong></div><p>Visualization only. No power, exposure time, wavelength, or device setting is
> represented.</p></div>
86     </aside>
87     </div>
88
89     <div class="tuning-panel" aria-labelledby="tune-title"><div class="tuning-intro"><span
> class="module-id">PL / LAST SAMPLE</span><h3 id="tune-title">Change the signal.</h3><p>Move a
> control to replace the final synthetic reading. Choose a scenario to reset it.</p><button
> type="button" id="reset-controls" class="text-button">Reset last sample ☒

```

index.html / continued

```
>   are arbitrary visualization tokens and cannot be translated into a safe treatment
> dose.</p><p>For a real wound, clinical evaluation remains essential. This research prototype is
> for technical demonstration only.</p><div class="boundary-links"><a href="./demo.html">Watch
> 86-second demo □</a><a href="./LIMITATIONS.md">Read limitations □</a><a
> href="./MODEL_CARD.md">Read model card □</a></div></div></div></section>
97   </main>
98   <footer class="footer"><span>PATCHLINE<sup>®</sup> / CONCEPT PROTOTYPE</span><span>Built by
> Rishik Rontala</span><a href="#top">Back to top ↑</a></footer>
99   </div>
100 </body>
101 </html>
```

demo.html

```

1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1">
6    <meta name="theme-color" content="#f4f1ea">
7    <meta name="author" content="Rishik Rontala">
8    <title>Patchline — 86-second demo</title>
9    <style>
10   :root{color-scheme:light;font-family:Arial,Helvetica,sans-serif;color:#100f0d;background:#f4
> f1ea}
11   *{box-sizing:border-box}body{margin:0}.wrap{width:min(1200px,calc(100% - 40px));margin:auto}
12   header{border-bottom:1px solid #d4cec1;padding:24px
> 0;display:flex;justify-content:space-between;gap:20px;align-items:center}
13   >
14   .brand{color:inherit;text-decoration:none;font-size:24px;font-weight:900;letter-spacing:-.07em}
15   .tag,.kicker,.note{font-family:ui-monospace,SFMono-Regular,Menlo,monospace;font-size:11px;le
> tter-spacing:.12em;text-transform:uppercase}
16   .tag{color:#da532c}main{padding:70px 0 90px}.kicker{color:#da532c}
17   h1{font-family:Georgia,serif;font-size:clamp(48px,9vw,120px);font-weight:400;line-height:.9;
> letter-spacing:-.055em;margin:25px 0 32px}
18   h1 em{font-weight:400}.intro{max-width:680px;font-size:20px;line-height:1.45;margin:0 0
> 40px}
19   video{width:100%;aspect-ratio:16/9;background:#100f0d;display:block;border:1px solid
> #c6c0b6}
20   >
21   .under{display:flex;justify-content:space-between;align-items:start;gap:24px;padding-top:20px}
22   .under
> p{margin:0;max-width:700px;line-height:1.5}.actions{display:flex;gap:20px;flex-wrap:wrap}
23   a{color:inherit}a:hover{color:#b44223}footer{border-top:1px solid #d4cec1;padding:25px 0
> 45px;display:flex;justify-content:space-between;gap:20px}
24   @media(max-width:700px){header,footer,.under{display:block}header .tag{margin-top:10px}.acti
> ons{margin-top:20px}main{padding-top:55px}h1{margin-bottom:24px}.intro{font-size:17px}}
25   </style>
26 </head>
27 <body>
28 <header class="wrap"><a class="brand" href="./index.html">PATCHLINE<span
> style="color:#da532c"></span></a><span class="tag">SIMULATION ONLY / RESEARCH
> PROTOTYPE</span></header>
29 <main class="wrap">
30 <div class="kicker">01 / WORKING PROTOTYPE FILM</div>
31 <h1>The signal<br><em>between visits.</em></h1>
32 <p class="intro">An 86-second walkthrough of four generated signals, a trained
> synthetic-pattern model, a sensor fault that stops the loop, and an evidence-linked handoff.</p>
33 <video controls preload="metadata" playsinline aria-label="Patchline prototype
> demonstration, silent, 86 seconds">
34 <source src="./submission/Patchline-Demo.mp4" type="video/mp4">
35 Your browser cannot play this video. <a href="./submission/Patchline-Demo.mp4">Download
> the MP4</a>.
36 </video>
37 <div class="under"><p>This film shows a software simulation. Patchline has no physical
> sensors, clinical validation, diagnosis, or treatment output.</p><div class="actions"><a
> href="./index.html#prototype">Explore prototype □</a><a
> href="./submission/Patchline-One-Page.pdf">Project brief □</a><a
> href="./submission/Patchline-Code.pdf">Source PDF □</a><a href="./submission/Patchline-Demo.mp4"
> download>Download video □</a></div></div>
38 </main>
39 <footer class="wrap"><span class="note">Built by Rishik Rontala</span><span class="note">No
> patient data / No treatment delivered</span></footer>
40 </body>
41 </html>

```

styles.css

```

1  :root{--paper:#f4f1ea;--ink:#151a18;--muted:#667670;--rule:#c9d0c9;--signal:#ce5b3a;--red:#a7423
> 6;--green:#627c6d;--panel:#e9eae2;--serif:"Iowan Old Style","Baskerville","Times New
> Roman",serif;--sans:"Helvetica
> Neue",Arial,sans-serif;--mono:"SFMono-Regular",Menlo,Monaco,Consolas,monospace}
2  *{box-sizing:border-box}
3  html{scroll-behavior:smooth}
4  body{margin:0;color:var(--ink);background:var(--paper);font-family:var(--sans);-webkit-font-smoo
> thing:antialiased}
5  button,input{font:inherit}
6  button{cursor:pointer}
7  a{color:inherit;text-decoration:none}
8  a:hover{text-decoration:none}
9  :focus-visible{outline:2px solid var(--signal);outline-offset:4px}
10 ::selection{background:#d9a690;color:var(--ink)}
11 .page-shell{max-width:1720px;margin:auto;overflow:hidden}
12 .skip-link{position:absolute;top:-60px;left:20px;padding:10px;background:var(--ink);color:white;
> z-index:100}
13 .skip-link:focus{top:10px}
14 .masthead{height:86px;padding:0
> 3.5vw;display:flex;align-items:center;justify-content:space-between;border-bottom:1px solid
> var(--rule)}
15 .brand{display:flex;align-items:center;gap:12px;font-family:var(--sans);font-size:21px;font-weig
> ht:800;letter-spacing:-.07em}
16 .brand-dot{color:var(--signal)}
17 .brand-mark{width:27px;height:27px;border:1px solid var(--ink);display:grid;grid-template-column
> s:repeat(3,1fr);align-items:center;gap:2px;padding:5px;transform:rotate(-8deg)}
18 .brand-mark i{background:var(--ink);height:100%;transform:skew(-8deg)}
19 .brand-mark i:nth-child(2){background:var(--signal)}
20 .masthead-center,.masthead-status,.section-index,.patch-panel-head,.patch-panel-foot,.hero-finep
> rint,.safety-band,.scenario-label,.scenario-number,.module-id,.module-header-right,.module-capti
> on,.quality-row,.score-label,.decision-eyebrow,.virtual-outputs-head,.virtual-outputs>p,.pipelin
> e-num,.report-meta,.metric-row,.event-log-title,.small-muted,.footer,.manual-control label,.manu
> al-value,.trace-unit,.trace-meta,.trace-axis,.claim-source,.live-tag,.print-button,.text-button{
> font-family:var(--mono);font-size:10px;font-weight:500;line-height:1.5;letter-spacing:.075em}
21 .masthead-center{color:var(--muted);letter-spacing:.19em}
22 .masthead-center span{padding:0 12px;color:var(--rule)}
23 .masthead-status{display:flex;align-items:center;gap:9px;color:var(--signal);font-weight:700;let
> ter-spacing:.1em}
24 .status-dot,.live-tag
> span{width:7px;height:7px;border-radius:50%;background:currentColor;display:inline-block}
25 .hero{padding:92px 3.5vw 54px;display:grid;grid-template-columns:minmax(0,1.2fr)
> minmax(390px,.8fr);gap:5vw;min-height:656px}
26 .hero-copy{position:relative;display:flex;flex-direction:column;align-items:flex-start;min-width
> :0}
27 .section-index{display:flex;align-items:center;gap:14px;color:var(--signal);font-weight:700;lett
> er-spacing:.13em}
28 .index-bar{display:block;width:20px;height:1px;background:currentColor}
29 .hero h1{font-family:var(--serif);font-weight:400;font-size:clamp(64px,7.6vw,125px);letter-spaci
> ng:-.068em;line-height:.88;margin:47px 0 33px}
30 .hero h1 em,.section-heading h2 em,.boundary-layout h2 em{font-weight:400}
31 .period{color:var(--signal)}
32 .hero-lede{font-size:19px;line-height:1.5;max-width:510px;margin:0;color:#35403b;letter-spacing:
> -.013em}
33 .hero-action-row{margin-top:auto;padding-top:44px;display:flex;gap:22px;align-items:center;flex-
> wrap:wrap}
34 .primary-link{background:var(--ink);color:var(--paper);padding:17px 19px 18px;min-width:199px;di
> splay:flex;justify-content:space-between;align-items:center;font-size:13px;font-weight:600;trans
> ition:background .2s ease,transform .2s ease}
35 .primary-link:hover{background:var(--signal);transform:translateY(-2px)}
36 .hero-fineprint{color:var(--muted);font-size:9px;letter-spacing:.02em}
37 .patch-panel{position:relative;min-height:480px;min-width:0;background:#e8ebe5;border:1px solid
> #b8c4bb;display:flex;flex-direction:column;overflow:hidden}

```

styles.css / continued

```

38 .patch-panel:before{content:"";position:absolute;inset:0;background-image:linear-gradient(#c7d2c
> 944 1px,transparent 1px),linear-gradient(90deg,#c7d2c944 1px,transparent
> 1px);background-size:31px 31px;mask-image:linear-gradient(to bottom,transparent 3%,black
> 30%,black 68%,transparent 95%);pointer-events:none}
39 .patch-panel-head,.patch-panel-foot{position:relative;z-index:2;display:flex;justify-content:spa
> ce-between;padding:20px 22px;color:#56675d;font-size:9px;font-weight:700;letter-spacing:.11em}
40 .patch-panel-foot{margin-top:auto;border-top:1px solid #b8c4bb;align-items:center}
41 .patch-visual{position:relative;min-height:380px;flex:1;display:flex;align-items:center;justify-
> content:center}
42 .patch-visual:before,.patch-visual:after{content:"";position:absolute;border:1px solid
> #bec9c0;border-radius:50%;width:400px;height:400px;opacity:.5}
43 .patch-visual:after{width:286px;height:286px;opacity:.65}
44 .patch-shape{width:clamp(250px,26vw,372px);height:190px;position:relative;display:flex;align-ite
> ms:center;filter:drop-shadow(0 18px 14px #64736819);transform:rotate(-19deg);z-index:1}
45 .patch-wing{height:156px;flex:1;background:#eee8dc;border:1px solid
> #aab9ad;position:relative;z-index:1}
46 .patch-wing:before{content:"";position:absolute;inset:7px;background-image:radial-gradient(#b9c6
> b7 1px,transparent 1.1px);background-size:11px 11px;opacity:.65}
47 .patch-wing-left{border-radius:38px 0 0 38px;border-right:0}
48 .patch-wing-right{border-radius:0 38px 38px 0;border-left:0}
49 .patch-core{width:49%;height:184px;flex:none;border-radius:25px;background:#d5ddd2;border:1px
> solid #879d8e;position:relative;z-index:2;display:flex;align-items:center;justify-content:center
> ;box-shadow:inset 0 0 0 7px #e2e8de}
50 .patch-core-grid{position:absolute;inset:14px;border-radius:16px;background:repeating-linear-gra
> dient(0deg,transparent 0 11px,#9eb2a150 11px 12px),repeating-linear-gradient(90deg,transparent 0
> 11px,#9eb2a150 11px 12px)}
51 .patch-center{width:108px;height:108px;background:#eaf0e8;border:1px solid #809889;border-radius
> :50%;position:relative;display:flex;align-items:center;justify-content:center}
52 .patch-ring{position:absolute;border-radius:50%;border:1px solid
> #a9b8aa;inset:12px}.patch-ring-b{inset:24px;border-color:#cad5c9}
53 .patch-cross{font-family:var(--mono);font-size:38px;color:#597365;font-weight:200;line-height:1}
54 .node{position:absolute;width:8px;height:8px;background:var(--signal);border:2px solid
> #f3f1e9;border-radius:50%;box-shadow:0 0 0 1px var(--signal)}
55 .node-t{top:3px;left:48px}.node-m{bottom:3px;left:48px}.node-p{left:2px;top:48px}.node-g{right:2
> px;top:48px}
56 .patch-label{position:absolute;z-index:3;display:flex;gap:8px;align-items:center;color:#496354;f
> ont-family:var(--mono);font-size:8px;font-weight:700;letter-spacing:.035em;white-space:nowrap}
57 .callout-line{width:36px;height:1px;background:#829d8d}
58 .patch-label-t{top:17%;left:8%}.patch-label-m{top:27%;right:4%;flex-direction:row-reverse}.patch
> -label-p{bottom:28%;left:5%}.patch-label-g{bottom:18%;right:5%;flex-direction:row-reverse}
59 .safety-band{padding:17px
> 3.5vw;display:flex;align-items:center;gap:15px;background:#e7e3d9;border-top:1px solid
> #d7d7cb;border-bottom:1px solid #d7d7cb;color:#4e5750;letter-spacing:.005em;font-size:11px}
60 .safety-band strong{color:var(--ink)}.safety-band a{margin-left:auto;text-decoration:underline;t
> ext-decoration-offset:3px;white-space:nowrap}.safety-sign{font-family:var(--serif);font-style:ita
> lic;border:1px solid #a87967;color:var(--signal);border-radius:50%;width:21px;height:21px;displa
> y:grid;place-items:center;flex:none;font-size:14px}
61 .prototype-section,.pipeline-section,.report-section,.boundaries{padding:100px 3.5vw 110px}
62 .section-heading{display:flex;justify-content:space-between;gap:30px;align-items:end;margin-bott
> om:44px}
63 .section-heading h2,.boundary-layout h2{font-family:var(--serif);font-size:clamp(42px,4.8vw,74px
> );font-weight:400;letter-spacing:-.055em;line-height:.98;margin:26px 0 0}
64 .section-heading>p{max-width:260px;color:#596760;font-size:14px;line-height:1.5;margin:0 0 4px}
65 .scenario-bar{display:grid;grid-template-columns:205px 1fr;border-top:1px solid
> var(--ink);border-bottom:1px solid var(--ink)}
66 .scenario-label{display:flex;align-items:center;justify-content:space-between;padding:0 20px 0
> 0;color:var(--muted);font-weight:700}.scenario-label span{font-size:21px;color:var(--signal)}
67 .scenario-controls{display:grid;grid-template-columns:repeat(3,1fr)}
68 .scenario-button{border:0;border-left:1px solid
> var(--rule);border-radius:0;background:transparent;min-height:82px;padding:15px 22px;display:fle
> x;align-items:center;gap:17px;text-align:left;font-size:16px;font-weight:500;transition:backgrou
> nd .2s ease,color .2s ease}

```

styles.css / continued

```

69 .scenario-button:hover{background:#ebe9e0}.scenario-button[aria-pressed="true"]{background:var(-
> -ink);color:var(--paper)}
70 .scenario-number{color:var(--signal);align-self:flex-start}.scenario-arrow{margin-left:auto;font
> -size:17px;color:var(--signal)}
71 .workbench{display:grid;grid-template-columns:minmax(0,1.53fr)
> minmax(355px,.77fr);margin-top:24px;border:1px solid var(--rule);min-height:550px}
72 .trace-panel{padding:29px 28px 20px;min-width:0}
73 .module-header{display:flex;justify-content:space-between;align-items:start;gap:12px;padding-bot
> tom:22px;border-bottom:1px solid var(--rule)}
74 .module-id{color:var(--signal);font-weight:700;letter-spacing:.12em}
75 .module-header h3,.tuning-intro
> h3{font-family:var(--serif);font-size:30px;letter-spacing:-.035em;line-height:1.1;margin:8px 0
> 0;font-weight:400}
76 .module-header-right{display:flex;align-items:center;gap:17px;color:var(--muted);font-size:9px;w
> hite-space:nowrap}
77 .live-tag{border:1px solid #a6b8a8;color:#55735d;padding:5px
> 7px;display:inline-flex;align-items:center;gap:6px;letter-spacing:.06em;font-size:9px}
78 .trace-grid{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:0 24px}
79 .trace-card{padding:20px 0 15px;border-bottom:1px solid var(--rule);min-width:0}
80 .trace-card:nth-last-child(-n+2){border-bottom:0}
81 .trace-head{display:flex;align-items:start;justify-content:space-between;gap:8px}
82 .trace-name{font-family:var(--mono);font-size:10px;color:#4e6055;letter-spacing:.04em;text-trans
> form:uppercase;font-weight:700}
83 .trace-reading{font-family:var(--serif);font-size:32px;letter-spacing:-.04em;line-height:1;margi
> n-top:6px;display:flex;align-items:baseline;gap:5px;font-variant-numeric:tabular-nums}
84 .trace-unit{font-size:9px;color:var(--muted);letter-spacing:.02em}
85 .trace-meta{color:var(--muted);font-size:9px;white-space:nowrap;text-align:right}
86 .trace-svg{display:block;width:100%;height:101px;margin-top:10px;overflow:visible}
87 .trace-gridline{stroke:#d3dad2;stroke-width:1;stroke-dasharray:2 4}.trace-baseline{stroke:#b9c4b
> a;stroke-width:1}.trace-path{fill:none;stroke:var(--green);stroke-width:2.2;stroke-linecap:round
> ;stroke-linejoin:round;vector-effect:non-scaling-stroke}.trace-card.is-changing
> .trace-path{stroke:var(--signal)}.trace-card.is-fault
> .trace-path{stroke:var(--red);stroke-dasharray:3 4}.trace-point{fill:var(--paper);stroke:var(--g
> reen);stroke-width:2}.trace-point.is-highlighted,.trace-card.is-changing .trace-point{stroke:var
> (--signal)}.trace-point.is-highlighted{r:5;stroke-width:3}.trace-gap{fill:var(--red)}
88 .trace-axis{display:flex;justify-content:space-between;color:#829087;font-size:8px;letter-spacin
> g:0}
89 .module-caption{font-size:9px;color:var(--muted);margin:20px 0
> 0;letter-spacing:0;line-height:1.6}
90 .decision-panel{background:var(--ink);color:var(--paper);padding:29px 28px
> 26px;min-width:0;display:flex;flex-direction:column}
91 .module-header-dark{border-color:#41504a}.module-header-dark
> .module-id{color:#d18168}.decision-step{font-family:var(--mono);font-size:10px;color:#9aafa2}
92 .quality-row{margin:22px 0 0;padding-bottom:16px;border-bottom:1px solid
> #41504a;display:flex;justify-content:space-between;color:#abbab0;font-size:10px}.quality-row
> strong{color:#b9d0bd;font-size:10px;letter-spacing:.08em}.decision-panel.is-fault .quality-row
> strong{color:#eca291}
93 .score-block{padding-top:30px}.score-label{color:#b3c4b7;display:flex;align-items:center;gap:6px
> ;font-size:9px;font-weight:700}.info-button{width:15px;height:15px;border:1px solid #73877b;back
> ground:none;color:#b3c4b7;border-radius:50%;padding:0;font-size:9px;line-height:13px}.info-butto
> n:hover{background:#324039}.score-value{font-family:var(--serif);font-size:clamp(74px,7vw,110px)
> ;letter-spacing:-.075em;line-height:1;margin:8px 0 0;font-variant-numeric:tabular-nums;white-spa
> ce:nowrap}.score-denominator{font-family:var(--mono);font-size:13px;letter-spacing:0;color:#8497
> 8a;margin-left:7px}.score-block p{font-size:11px;line-height:1.5;color:#a2b1a5;margin:4px 0
> 0;max-width:260px}
94 .decision-outcome{margin-top:27px;border-top:1px solid
> #41504a;padding-top:20px}.decision-eyebrow{color:#afc0b1;font-size:9px}.decision-outcome h4{font
> -family:var(--serif);font-size:30px;font-weight:400;line-height:1.05;letter-spacing:-.035em;marg
> in:9px 0}.decision-outcome p{font-size:12px;color:#a4b5a8;line-height:1.5;margin:0}
95 .virtual-outputs{margin-top:auto;padding-top:23px}.virtual-outputs-head{border-top:1px solid #41
> 504a;padding-top:16px;margin-bottom:13px;display:flex;justify-content:space-between;gap:8px;colo
> r:#b0c0b3;font-size:9px;letter-spacing:.05em}.virtual-outputs-head

```

styles.css / continued

```

> span:last-child{color:#d8876c}.output-line{display:grid;grid-template-columns:92px 1fr
> 41px;gap:10px;align-items:center;margin:12px 0;font-size:12px}.output-line strong{font-family:va
> r(--mono);font-size:10px;text-align:right;font-weight:500;color:#d8e4d9}.level-track{display:fle
> x;gap:4px}.level-track i{height:8px;flex:1;background:#46564c;display:block}.level-track i.on{ba
> ckground:var(--signal)}.virtual-outputs>p{color:#88998d;font-size:9px;letter-spacing:0;line-heig
> ht:1.5;margin:13px 0 0}
96 .tuning-panel{display:grid;grid-template-columns:280px 1fr;gap:35px;padding:34px 0
> 0;border-bottom:1px solid var(--rule)}
97 .tuning-intro h3{font-size:29px}.tuning-intro
> p{font-size:12px;color:var(--muted);line-height:1.5;margin:11px 0
> 13px}.text-button{padding:0;border:0;border-bottom:1px solid var(--signal);background:none;color
> :var(--signal);font-weight:700;letter-spacing:0;line-height:1.5}.text-button:hover{color:var(--i
> nk);border-color:var(--ink)}
98 .manual-controls{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:0
> 28px}.manual-control{padding:0 0 24px}.manual-control label{display:flex;justify-content:space-b
> etween;gap:10px;font-size:9px;color:#56665c;letter-spacing:.04em;text-transform:uppercase;font-w
> eight:700}.manual-value{font-size:11px;color:var(--ink);white-space:nowrap;letter-spacing:0}.man
> ual-control input[type="range"]{appearance:none;width:100%;height:17px;margin:15px 0
> 4px;background:transparent;cursor:pointer}.manual-control
> input[type="range"]::-webkit-slider-runable-track{height:2px;background:#aebcb0}.manual-control
> input[type="range"]::-moz-range-track{height:2px;background:#aebcb0}.manual-control input[type=
> "range"]::-webkit-slider-thumb{appearance:none;width:14px;height:14px;border-radius:50%;backgrou
> nd:var(--signal);border:3px solid var(--paper);box-shadow:0 0 0 1px
> var(--signal);margin-top:-6px}.manual-control input[type="range"]::-moz-range-thumb{width:9px;he
> ight:9px;border-radius:50%;background:var(--signal);border:3px solid var(--paper);box-shadow:0 0
> 0 1px var(--signal)}.range-ends{display:flex;justify-content:space-between;color:#89968b;font-f
> amily:var(--mono);font-size:8px}
99 .pipeline-section{background:#e7e9e2}.section-heading.compact{margin-bottom:60px}.pipeline-grid{
> display:grid;grid-template-columns:repeat(4,1fr);border-top:1px solid #9eaeae;border-bottom:1px
> solid #9eaeae}.pipeline-item{min-height:265px;padding:18px 22px 25px;border-right:1px solid #acb
> 9ad;position:relative}.pipeline-item:first-child{padding-left:0}.pipeline-item:last-child{border
> -right:0}.pipeline-num{color:var(--signal);font-size:10px}.pipeline-icon{font-family:var(--serif
> );font-size:47px;line-height:1;color:#61786a;margin-top:30px;height:51px}.pipeline-item
> h3{font-family:var(--serif);font-size:25px;font-weight:400;letter-spacing:-.025em;margin:13px 0
> 9px}.pipeline-item p{font-size:12px;color:#526158;line-height:1.5;max-width:210px;margin:0}
100 .report-section{padding-bottom:122px}.print-button{background:transparent;border:1px solid
> var(--ink);padding:14px 17px;font-size:10px;font-weight:700;white-space:nowrap;display:flex;gap:
> 28px}.print-button:hover{background:var(--ink);color:var(--paper)}
101 .report-layout{display:grid;grid-template-columns:1.35fr .65fr;border-top:1px solid
> var(--ink);border-bottom:1px solid var(--ink)}.report-main{padding:26px 6vw 40px
> 0;border-right:1px solid var(--rule)}.report-meta{display:flex;justify-content:space-between;gap
> :20px;color:var(--signal);font-size:9px;font-weight:700}.report-main h3{font-family:var(--serif)
> ;font-size:clamp(36px,4vw,62px);font-weight:400;letter-spacing:-.048em;line-height:1.05;margin:3
> 4px 0 16px}.report-main>p{max-width:670px;font-size:15px;color:#48574d;line-height:1.6;margin:0
> 0 35px}.claim-list{display:flex;flex-direction:column;border-top:1px solid
> var(--rule)}.claim-row{padding:16px 0;border-bottom:1px solid
> var(--rule);display:grid;grid-template-columns:1fr auto;gap:16px;align-items:center}.claim-row>s
> pan{font-size:12px;line-height:1.45}.claim-source{padding:7px 8px;border:1px solid #adbdb0;backg
> round:transparent;color:#527363;font-size:8px;font-weight:700;white-space:nowrap}.claim-source:h
> over,.claim-source:focus-visible{background:var(--green);color:white}.report-side{padding:28px 0
> 30px 32px}.report-side>p{font-size:13px;line-height:1.55;color:#536057;margin:17px 0
> 25px}.report-side>p strong{color:var(--ink)}.metric-row{border-top:1px solid
> var(--rule);padding:10px 0;display:flex;justify-content:space-between;gap:12px;align-items:cente
> r;color:#5d6d61;font-size:9px}.metric-row strong{font-family:var(--serif);font-size:24px;font-we
> ight:400;letter-spacing:-.03em;color:var(--ink)}.report-side .small-muted{margin:8px 0 31px;colo
> r:#7b887c;font-size:9px;line-height:1.5;letter-spacing:0}.event-log-title{font-weight:700;color:
> var(--signal);font-size:9px}.event-log{list-style:none;padding:0;margin:13px 0 0;border-top:1px
> solid var(--rule)}.event-log li{padding:8px 0;border-bottom:1px solid
> var(--rule);font-family:var(--mono);font-size:9px;line-height:1.45;color:#53635a}.event-log
> li:before{content:"□";color:var(--signal);padding-right:8px}
102 .boundaries{background:#26302a;color:var(--paper);padding-bottom:115px}.boundaries
> .section-index{color:#d38167}.boundary-layout{display:grid;grid-template-columns:1fr

```

styles.css / continued

```

> 1fr;gap:5vw;align-items:start}.boundary-layout h2{font-size:clamp(65px,6vw,100px);margin-top:28p
> x}.boundary-layout>div{padding-top:31px}.boundary-layout
> p{font-size:17px;color:#c2d0c3;line-height:1.55;margin:0 0 18px;max-width:570px}.boundary-links{
> display:flex;flex-wrap:wrap;gap:25px;margin-top:37px}.boundary-links
> a{font-family:var(--mono);font-size:10px;color:#e0a18c;border-bottom:1px solid
> #e0a18c;padding-bottom:5px}.boundary-links a:hover{color:white;border-color:white}.footer{displa
> y:flex;justify-content:space-between;gap:20px;align-items:center;padding:27px
> 3.5vw;font-size:9px;color:#536257;letter-spacing:.09em}.footer sup{font-size:7px}.footer
> a:hover{color:var(--signal)}
103 @media(max-width:1100px){.hero{grid-template-columns:1fr .83fr;gap:2.5vw}.hero
> h1{font-size:clamp(56px,7vw,88px)}.patch-label{font-size:7px}.patch-label
> .callout-line{width:22px}.workbench{grid-template-columns:1.3fr .8fr}.trace-grid{gap:0
> 14px}.scenario-bar{grid-template-columns:150px 1fr}.scenario-button{gap:7px;padding:12px
> 14px;font-size:14px}}
104 @media(max-width:850px){.masthead-center{display:none}.hero{grid-template-columns:1fr;min-height
> :auto;padding-top:65px}.hero-copy{min-height:470px}.hero h1{font-size:clamp(66px,12vw,105px)}.pa
> tch-panel{min-height:420px}.workbench{grid-template-columns:1fr}.decision-panel{min-height:420px
> }.score-value{font-size:94px}.decision-panel{display:grid;grid-template-columns:1fr 1fr;gap:0
> 28px}.decision-panel .module-header,.decision-panel
> .quality-row{grid-column:1/-1}.decision-panel .virtual-outputs{grid-column:1/-1}.score-block{pad
> ding-top:24px}.decision-outcome{margin-top:24px}.tuning-panel{grid-template-columns:1fr;gap:22px
> }.pipeline-grid{grid-template-columns:repeat(2,1fr)}.pipeline-item:nth-child(2){border-right:0}.
> pipeline-item:nth-child(-n+2){border-bottom:1px solid
> #acb9ad}.report-layout{grid-template-columns:1fr}.report-main{border-right:0;border-bottom:1px
> solid var(--rule);padding-right:0}.report-side{padding-left:0}.boundary-layout{gap:20px}}
105 @media(max-width:620px){.masthead{height:66px;padding:0
> 20px}.brand{font-size:19px}.masthead-status{font-size:8px}.hero{padding:53px 20px
> 32px;grid-template-columns:minmax(0,1fr)}.hero-copy{min-height:430px}.hero
> h1{font-size:clamp(49px,13.5vw,72px);margin:40px 0 24px}.hero-lede{font-size:16px}.hero-action-r
> ow{gap:14px;padding-top:30px}.hero-fineprint{width:100%}.patch-panel{min-height:365px}.patch-vis
> ual{min-height:300px}.patch-shape{height:150px;width:270px}.patch-wing{height:120px}.patch-core{
> height:147px}.patch-label{font-size:6px}.patch-label-t{top:13%;left:4%}.patch-label-m{top:22%;ri
> ght:3%}.patch-label-p{bottom:23%;left:2%}.patch-label-g{bottom:12%;right:2%}.patch-label .callou
> t-line{width:13px}.patch-panel-head,.patch-panel-foot{padding:14px;font-size:7px}.safety-band{pa
> dding:15px 20px;align-items:start;flex-wrap:wrap;font-size:10px}.safety-band
> a{margin-left:35px}.prototype-section,.pipeline-section,.report-section,.boundaries{padding:72px
> 20px 78px}.section-heading{display:block;margin-bottom:31px}.section-heading
> h2{font-size:clamp(45px,11vw,68px)}.section-heading>p{margin:20px 0 0;max-width:340px}.scenario-
> bar{display:block}.scenario-label{height:40px}.scenario-controls{display:block}.scenario-button{
> width:100%;min-height:59px;border-left:0;border-top:1px solid var(--rule);font-size:14px}.scenar
> io-number{align-self:center}.workbench{margin-top:15px}.trace-panel,.decision-panel{padding:22px
> 19px}.trace-grid{grid-template-columns:1fr}.trace-card:nth-last-child(2){border-bottom:1px
> solid var(--rule)}.trace-svg{height:86px}.module-header-right{gap:7px}.module-header
> h3{font-size:27px}.decision-panel{display:block}.decision-outcome{border-top:1px solid #41504a;m
> argin-top:24px;padding-top:20px}.virtual-outputs{margin-top:25px}.manual-controls{grid-template-
> columns:1fr}.manual-control{padding-bottom:17px}.tuning-panel{padding-top:28px}.pipeline-grid{gr
> id-template-columns:1fr}.pipeline-item{min-height:190px;border-right:0;border-bottom:1px solid
> #acb9ad;padding:14px 0 22px}.pipeline-item:last-child{border-bottom:0}.pipeline-icon{font-size:3
> 3px;height:35px;margin-top:16px}.pipeline-item h3{margin-top:5px}.pipeline-item p{max-width:340p
> x}.section-heading.compact{margin-bottom:35px}.print-button{margin-top:28px}.report-main h3{font
> -size:42px}.report-meta{font-size:8px}.claim-row{grid-template-columns:1fr}.claim-source{justify
> -self:start}.boundary-layout{grid-template-columns:1fr}.boundary-layout
> h2{font-size:66px}.boundary-layout>div{padding-top:12px}.boundary-layout
> p{font-size:15px}.footer{padding:23px 20px;flex-wrap:wrap;gap:10px}.footer
> span:nth-child(2){order:3;width:100%}}
106 @media(prefers-reduced-motion:reduce){html{scroll-behavior:auto}*,*:before,*:after{animation-dur
> ation:.01ms!important;transition-duration:.01ms!important}}
107 @media(max-width:620px){.module-header-right{flex-direction:column;align-items:flex-end;gap:5px}
> .module-header h3{max-width:200px}}
108 @media print{@page{size:A4;margin:12mm}html,body{background:#fff;color:#000}body{font-size:9pt}.
> page-shell{overflow:visible}.masthead,.hero,.safety-band,.prototype-section,.pipeline-section,.b
> oundaries,.footer,.section-heading .print-button{display:none!important}.report-section{display:

```

styles.css / continued

```
> block!important;padding:0!important}.report-section .section-heading{margin:0 0
> 8mm!important}.report-section .section-index{font-size:7pt}.report-section
> h2{font-size:28pt!important;margin:5mm
> 0!important}.report-layout{display:grid!important;grid-template-columns:1.25fr
> .75fr!important}.report-main{padding:6mm 7mm 0 0!important;border-right:1px solid
> #bbb!important;border-bottom:0!important}.report-main h3{font-size:25pt!important;margin:8mm 0 4
> mm!important}.report-main>p{font-size:9pt!important;margin-bottom:8mm!important}.report-side{pad
> ding:6mm 0 0 6mm!important}.report-side>p{font-size:8pt!important}.claim-row{display:grid!import
> ant;grid-template-columns:1fr auto!important;padding:3mm 0!important}.claim-source{border:0!impo
> rtant;padding:0!important;font-size:6pt!important}.report-section,.report-layout,.report-main,.r
> eport-side{break-inside:avoid}.event-log
> li{font-size:7pt}.report-section:after{content:"Patchline is a synthetic system-design
> demonstration. It is not a medical device or a treatment recommendation. Built by Rishik Rontala
> .";display:block;font-family:var(--mono);font-size:6pt;margin-top:12mm;color:#555;border-top:1px
> solid #aaa;padding-top:3mm}}
```

app.js

```
1  import { generateScenario, trainSyntheticModel, analyze } from "./engine.js";
2
3  const channels = [
4    { key: "temperature", name: "Patch temperature", unit: "°C", min: 20, max: 45, step: 0.01,
5    > digits: 2 },
6    { key: "moisture", name: "Dressing wetness", unit: "%", min: 0, max: 100, step: 0.01, digits:
7    > 1 },
8    { key: "ph", name: "Wound-fluid pH", unit: "pH", min: 4, max: 10, step: 0.01, digits: 2 },
9    { key: "glucose", name: "Wound-fluid glucose*", unit: "AU", min: 0, max: 100, step: 0.01,
10   > digits: 1 },
11  ];
12
13  const $ = (selector) => document.querySelector(selector);
14  const svgNS = "http://www.w3.org/2000/svg";
15  const model = trainSyntheticModel();
16  let scenario;
17  let samples;
18  let result;
19  let highlightedSamples = new Set();
20  let hasManualEdit = false;
21
22  function format(value, digits = 1) {
23    return typeof value === "number" && Number.isFinite(value) ? value.toFixed(digits) : "-";
24  }
25
26  function percent(value) {
27    return typeof value === "number" && Number.isFinite(value) ? `${Math.round(value * 100)}%` :
28    > "-";
29  }
30
31  function svgNode(name, attributes = {}) {
32    const element = document.createElementNS(svgNS, name);
33    for (const [key, value] of Object.entries(attributes)) element.setAttribute(key,
34    > String(value));
35    return element;
36  }
37
38  function sampleNumber(index) {
39    return `S${String(index + 1).padStart(2, "0")}`;
40  }
41
42  function currentScore() {
43    return result.patternScore == null ? "-" : String(Math.round(result.patternScore * 100));
44  }
45
46  function loadScenario(key) {
47    scenario = generateScenario(key);
48    samples = scenario.samples.map((sample) => ({ ...sample }));
49    highlightedSamples = new Set();
50    $("#trace-grid").removeAttribute("aria-label");
51    hasManualEdit = false;
52    document.querySelectorAll(".scenario-button").forEach((button) => {
53      button.setAttribute("aria-pressed", String(button.dataset.scenario === key));
54    });
55    renderControls();
56    update();
57  }
58
59  function update() {
60    result = analyze(samples, model);
61    renderTraces();
62    renderDecision();
63  }
```

app.js / continued

```
58   renderReport();
59   renderLog();
60 }
61
62 function renderControls() {
63   const container = $("#manual-controls");
64   container.replaceChildren();
65   const latest = samples.at(-1);
66   for (const channel of channels) {
67     const wrapper = document.createElement("div");
68     wrapper.className = "manual-control";
69     const label = document.createElement("label");
70     const labelName = document.createElement("span");
71     labelName.textContent = channel.name;
72     const value = document.createElement("strong");
73     value.className = "manual-value";
74     value.textContent = latest[channel.key] == null ? "MISSING" : `${format(latest[channel.key],
75 > channel.digits)} ${channel.unit}`;
76     const id = `manual-${channel.key}`;
77     label.htmlFor = id;
78     label.append(labelName, value);
79     const input = document.createElement("input");
80     input.id = id;
81     input.type = "range";
82     input.min = String(channel.min);
83     input.max = String(channel.max);
84     input.step = String(channel.step);
85     input.value = String(latest[channel.key] == null ? (channel.min + channel.max) / 2 :
86 > latest[channel.key]);
87     input.setAttribute("aria-label", `Last synthetic ${channel.name.toLowerCase()} reading`);
88     input.addEventListener("input", () => {
89       const reading = Number(input.value);
90       samples[samples.length - 1][channel.key] = reading;
91       value.textContent = `${format(reading, channel.digits)} ${channel.unit}`;
92       hasManualEdit = true;
93       update();
94     });
95     const ends = document.createElement("div");
96     ends.className = "range-ends";
97     const low = document.createElement("span");
98     const high = document.createElement("span");
99     low.textContent = String(channel.min);
100    high.textContent = `${channel.max} ${channel.unit}`;
101    ends.append(low, high);
102    wrapper.append(label, input, ends);
103    container.append(wrapper);
104  }
105
106 function createTraceSvg(channel, evidence) {
107   const width = 320;
108   const height = 93;
109   const top = 8;
110   const bottom = 8;
111   const x = (index) => 5 + (index / (samples.length - 1)) * (width - 10);
112   const numeric = samples.map((sample) => sample[channel.key]).filter((value) => typeof value
113 > == "number" && Number.isFinite(value));
114   const rawMin = Math.min(...numeric);
115   const rawMax = Math.max(...numeric);
116   const spread = Math.max(rawMax - rawMin, channel.key == "ph" ? 0.25 : channel.key ==
117 > "temperature" ? 0.6 : 5);
118   const min = rawMin - spread * 0.18;
```

app.js / continued

```

116   const max = rawMax + spread * 0.18;
117   const y = (value) => top + (1 - (value - min) / (max - min)) * (height - top - bottom);
118   const svg = svgNode("svg", { class: "trace-svg", viewBox: `0 0 ${width} ${height}`, role:
> "img", "aria-label": `${channel.name} across 24 synthetic samples` });
119   svg.append(svgNode("line", { x1: 0, x2: width, y1: 28, y2: 28, class: "trace-gridline" }));
120   svg.append(svgNode("line", { x1: 0, x2: width, y1: 64, y2: 64, class: "trace-gridline" }));
121   svg.append(svgNode("line", { x1: 0, x2: width, y1: height - 1, y2: height - 1, class:
> "trace-baseline" }));
122   let path = "";
123   let started = false;
124   samples.forEach((sample, index) => {
125     const value = sample[channel.key];
126     if (typeof value !== "number" || !Number.isFinite(value)) {
127       started = false;
128       svg.append(svgNode("path", { d: `M ${x(index) - 4} 45 l 8 8 M ${x(index) + 4} 45 l -8 8`,
> class: `trace-gap${highlightedSamples.has(index) ? " is-highlighted" : ""}`, stroke:
> highlightedSamples.has(index) ? "#ce5b3a" : "#a74236", "stroke-width":
> highlightedSamples.has(index) ? 3 : 1.7, fill: "none" }));
129     return;
130   }
131   path += `${started ? " L" : " M"} ${x(index).toFixed(1)} ${y(value).toFixed(1)}`;
132   started = true;
133 });
134 svg.append(svgNode("path", { d: path, class: "trace-path" }));
135 samples.forEach((sample, index) => {
136   const value = sample[channel.key];
137   if (typeof value !== "number" || !Number.isFinite(value)) return;
138   if (index !== samples.length - 1 && !evidence.includes(index) &&
> !highlightedSamples.has(index)) return;
139   const point = svgNode("circle", { cx: x(index), cy: y(value), r:
> highlightedSamples.has(index) ? 5 : 3.1, class: `trace-point${highlightedSamples.has(index) ? "
> is-highlighted" : ""}` });
140   const title = svgNode("title");
141   title.textContent = `${sampleNumber(index)}: ${format(value, channel.digits)}
> ${channel.unit}, synthetic`;
142   point.append(title);
143   svg.append(point);
144 });
145 return svg;
146 }
147
148 function renderTraces() {
149   const grid = $("#trace-grid");
150   grid.replaceChildren();
151   $("#sample-count").textContent = `${samples.length} virtual samples`;
152   const evidence = result.evidenceIndices || [];
153   for (const channel of channels) {
154     const card = document.createElement("div");
155     card.className = `trace-card ${result.quality.valid ? (result.decision.action === "review" ?
> "is-changing" : "") : "is-fault"}`;
156     card.id = `trace-${channel.key}`;
157     const head = document.createElement("div");
158     head.className = "trace-head";
159     const left = document.createElement("div");
160     const name = document.createElement("div");
161     name.className = "trace-name";
162     name.textContent = channel.name;
163     const reading = document.createElement("div");
164     reading.className = "trace-reading";
165     const numeric = document.createElement("span");
166     numeric.textContent = format(result.latest[channel.key], channel.digits);
167     const unit = document.createElement("span");

```

app.js / continued

```

168     unit.className = "trace-unit";
169     unit.textContent = channel.unit;
170     reading.append(numeric, unit);
171     left.append(name, reading);
172     const meta = document.createElement("span");
173     meta.className = "trace-meta";
174     meta.textContent = highlightedSamples.size === 1
175       ? `${sampleNumber([...highlightedSamples][0])} highlighted`
176       : highlightedSamples.size > 1 ? `${highlightedSamples.size} samples highlighted` : "LATEST
> / SYNTHETIC";
177     head.append(left, meta);
178     const axis = document.createElement("div");
179     axis.className = "trace-axis";
180     const first = document.createElement("span");
181     const last = document.createElement("span");
182     first.textContent = "S01 / START";
183     last.textContent = "S24 / LATEST";
184     axis.append(first, last);
185     card.append(head, createTraceSvg(channel, evidence), axis);
186     grid.append(card);
187   }
188 }
189
190 function renderDecision() {
191   const valid = result.quality.valid;
192   $(".decision-panel").classList.toggle("is-fault", !valid);
193   $("#quality-value").textContent = valid ? "PASS / 24 SAMPLES" : `HOLD /
> ${result.quality.flags.length} FLAG${result.quality.flags.length === 1 ? "" : "S"}`;
194   $("#score-value").textContent = currentScore();
195   $("#score-caption").textContent = valid
196     ? "Classifier output on generated traces. Not an infection probability."
197     : "Score withheld: the synthetic sensor-quality gate found invalid data.";
198   $("#decision-label").textContent = result.decision.label;
199   $("#decision-rationale").textContent = result.decision.rationale;
200   for (const [key, prefix] of [["ultrasoundLevel", "ultrasound"], ["violetLevel", "violet"]]) {
201     const level = result.decision[key];
202     `${prefix}-value`.textContent = `${level} / 3`;
203     document.querySelectorAll(`${prefix}-track i`).forEach((segment, index) =>
> segment.classList.toggle("on", index < level));
204   }
205 }
206
207 function evidenceButton(indices, text) {
208   const button = document.createElement("button");
209   button.type = "button";
210   button.className = "claim-source";
211   const ordered = [...new Set(indices)].filter((index) => Number.isInteger(index) && index >= 0
> && index < samples.length);
212   button.textContent = text || (ordered.length === 1 ? `VIEW ${sampleNumber(ordered[0])} ` :
> `VIEW ${ordered.map(sampleNumber).join(", ")} `);
213   button.addEventListener("click", () => {
214     highlightedSamples = new Set(ordered);
215     renderTraces();
216     $("#trace-grid").scrollIntoView({ behavior: window.matchMedia("(prefers-reduced-motion:
> reduce)").matches ? "auto" : "smooth", block: "center" });
217     if (highlightedSamples.size) $("#trace-grid").setAttribute("aria-label",
> `${ordered.map(sampleNumber).join(" and ")} highlighted in the synthetic traces`);
218   });
219   return button;
220 }
221
222 function addClaim(container, description, indices, sourceText) {

```

app.js / continued

```

223   const row = document.createElement("div");
224   row.className = "claim-row";
225   const text = document.createElement("span");
226   text.textContent = description;
227   row.append(text, evidenceButton(indices, sourceText));
228   container.append(row);
229 }
230
231 function renderReport() {
232   $("#report-scenario").textContent = `${scenario.id.toUpperCase()} / ${hasManualEdit ?
  > "MODIFIED" : "ORIGINAL"}`;
233   $("#report-headline").textContent = result.quality.valid
234   ? result.decision.action === "review" ? "Changing pattern, human review prepared." : "Stable
  > pattern, trace recorded."
235   : "Sensor fault, score withheld.";
236   $("#report-summary").textContent = `${scenario.description} ${result.decision.rationale}`;
237   $("#train-count").textContent = String(model.trainCount);
238   $("#test-count").textContent = String(model.testCount);
239   $("#metric-accuracy").textContent = percent(model.metrics.accuracy);
240   $("#metric-recall").textContent = percent(model.metrics.recall);
241   const claims = $("#claim-list");
242   claims.replaceChildren();
243   const latestIndex = samples.length - 1;
244   for (const channel of channels) {
245     const value = result.latest[channel.key];
246     addClaim(claims, `Latest synthetic ${channel.name.toLowerCase(): ${value == null ?
  > "missing" : `${format(value, channel.digits)} ${channel.unit}`}.`, [latestIndex]);
247   }
248   if (!result.quality.valid) {
249     result.quality.flags.slice(0, 3).forEach((flag, index) => addClaim(claims, flag,
  > [result.evidenceIndices[index] ?? latestIndex]));
250     addClaim(claims, "Quality gate held the model score and both virtual output levels at
  > zero.", result.evidenceIndices, "VIEW FLAGGED SAMPLES □");
251   } else {
252     addClaim(claims, `Synthetic pattern score: ${currentScore()}/100. This is not infection
  > risk.`, result.evidenceIndices, "VIEW EVIDENCE □");
253     addClaim(claims, `Orchestrator action: ${result.decision.label}. Virtual ultrasound
  > ${result.decision.ultrasoundLevel}/3 and violet light ${result.decision.violetLevel}/3 abstract
  > tokens; no physical dose.`, result.evidenceIndices, "VIEW EVIDENCE □");
254   }
255 }
256
257 function renderLog() {
258   const log = $("#event-log");
259   log.replaceChildren();
260   const entries = [
261     `BOOT: ${scenario.id.toUpperCase()} seed loaded locally`,
262     `SAMPLE: ${samples.length} synthetic frames / 4 channels`,
263     result.quality.valid ? "QC: all channels pass range + spike checks" : `QC: hold /
  > ${result.quality.flags.length} flagged reading${result.quality.flags.length === 1 ? "" : "s"}`,
264     result.quality.valid ? `MODEL: ${currentScore()}/100 synthetic pattern score` : "MODEL:
  > blocked by quality gate",
265     `POLICY: ${result.decision.action.toUpperCase()} / virtual output
  > ${result.decision.ultrasoundLevel}, ${result.decision.violetLevel}`,
266   ];
267   if (hasManualEdit) entries.splice(2, 0, "OVERRIDE: final synthetic frame adjusted by
  > visitor");
268   for (const entry of entries) {
269     const item = document.createElement("li");
270     item.textContent = entry;
271     log.append(item);
272   }

```

app.js / continued

```
273 }
274
275 document.querySelectorAll(".scenario-button").forEach((button) =>
  > button.addEventListener("click", () => loadScenario(button.dataset.scenario)));
276 $("#reset-controls").addEventListener("click", () => loadScenario(scenario.id));
277 $("#print-report").addEventListener("click", () => window.print());
278 $("#score-info").addEventListener("click", () => {
279   $("#score-caption").textContent = "The classifier was fitted on generated stable/changing
  > patterns and checked on separate generated patterns. This score is not calibrated to infection
  > and must never guide care.";
280 });
281
282 loadScenario("stable");
```

engine.js

```

1  // Patchline's entire data and control path is a reproducible DIGITAL TWIN.
2  // These levels are display tokens, never device settings or treatment doses.
3
4  const CHANNELS = ["temperature", "moisture", "ph", "glucose"];
5  const FEATURE_NAMES = ["temperatureTrend", "moistureTrend", "phTrend", "glucoseTrend"];
6  const FEATURE_SCALES = [2, 20, 0.8, 20];
7  const LIMITS = {
8    temperature: [20, 45], // local patch/surface °C, synthetic
9    moisture: [0, 100], // dressing wetness %, synthetic
10   ph: [4, 10], // wound-fluid pH, synthetic
11   glucose: [0, 100], // experimental wound-fluid reading, arbitrary units
12 };
13
14 function random(seed) {
15   let state = seed >>> 0;
16   return () => {
17     state = (state + 0x6d2b79f5) >>> 0;
18     let value = state;
19     value = Math.imul(value ^ (value >>> 15), value | 1);
20     value ^= value + Math.imul(value ^ (value >>> 7), value | 61);
21     return ((value ^ (value >>> 14)) >>> 0) / 4294967296;
22   };
23 }
24
25 const round = (value, places = 2) => Number(value.toFixed(places));
26 const average = (values) => values.reduce((sum, value) => sum + value, 0) / values.length;
27
28 function makeTrace(seed, changing, abrupt = false, boundary = false) {
29   const next = random(seed);
30   const baseline = {
31     temperature: 32.0 + next() * 1.7,
32     moisture: 35 + next() * 13,
33     ph: 5.45 + next() * 0.85,
34     glucose: 28 + next() * 18,
35   };
36   // Borderline cases overlap across labels on purpose. This avoids a
37   // misleadingly perfect held-out result from trivially separable examples.
38   const amplitude = boundary
39     ? {
40       temperature: 0.2 + next() * 0.65,
41       moisture: 2 + next() * 7,
42       ph: 0.1 + next() * 0.28,
43       glucose: 3 + next() * 8,
44     }
45     : changing
46     ? {
47       temperature: 1.0 + next() * 1.45,
48       moisture: 10 + next() * 16,
49       ph: 0.38 + next() * 0.68,
50       glucose: 12 + next() * 22,
51     }
52     : {
53       temperature: (next() - 0.5) * 0.36,
54       moisture: (next() - 0.5) * 3,
55       ph: (next() - 0.5) * 0.14,
56       glucose: (next() - 0.5) * 3,
57     };
58   const onset = 10 + Math.floor(next() * 6);
59   const samples = [];
60
61   for (let t = 0; t < 24; t += 1) {
62     const progress = abrupt

```

engine.js / continued

```

63     ? (t < onset ? 0 : 0.64 + 0.36 * (t - onset) / (23 - onset))
64     : Math.pow(t / 23, 1.3);
65     const cycle = Math.sin((t / 23) * Math.PI * 2 + seed % 9) * 0.04;
66     samples.push({
67       t,
68       temperature: round(baseline.temperature + amplitude.temperature * progress + cycle +
69 > (next() - 0.5) * 0.13),
69       moisture: round(baseline.moisture + amplitude.moisture * progress + cycle * 4 + (next() -
70 > 0.5) * 1.1),
70       ph: round(baseline.ph + amplitude.ph * progress + cycle * 0.15 + (next() - 0.5) * 0.055),
71       glucose: round(baseline.glucose + amplitude.glucose * progress + cycle * 4 + (next() -
72 > 0.5) * 1.3),
72     });
73   }
74   return samples;
75 }
76
77 export function generateScenario(key) {
78   switch (key) {
79     case "stable":
80       return {
81         id: "stable",
82         label: "Stable synthetic trace",
83         description: "Four reproducible generated channels with small ordinary variation.",
84         samples: makeTrace(0x71a2c, false),
85       };
86     case "change":
87       return {
88         id: "change",
89         label: "Changing synthetic pattern",
90         description: "A generated multi-channel shift for demonstrating model and review flow;
91 > it is not an infection case.",
92         samples: makeTrace(0xc11a9e, true),
93       };
94     case "fault": {
95       const samples = makeTrace(0xfa017, true);
96       samples[21].ph = 12.2;
97       samples[23].glucose = null;
98       return {
99         id: "fault",
100        label: "Sensor fault",
101        description: "A generated trace with an implausible pH reading and missing final glucose
102 > reading; the quality gate must hold.",
103        samples,
104      };
105    }
106    default:
107      throw new RangeError(`Unknown synthetic scenario: ${String(key)}`);
108  }
109 }
110
111 function features(samples) {
112   const head = samples.slice(0, 6);
113   const tail = samples.slice(-6);
114   return CHANNELS.map((name, index) =>
115     (average(tail.map((sample) => sample[name])) - average(head.map((sample) => sample[name])))
116     > / FEATURE_SCALES[index],
117   );
118 }
119
120 function sigmoid(value) {
121   if (value >= 0) return 1 / (1 + Math.exp(-value));

```

engine.js / continued

```
119   const exponential = Math.exp(value);
120   return exponential / (1 + exponential);
121 }
122
123 function makeDataset(count, seedBase) {
124   return Array.from({ length: count }, (_, index) => {
125     const changing = index % 2 === 1;
126     const seed = (seedBase + Math.imul(index + 1, 0x9e3779b1)) >>> 0;
127     return {
128       x: features(makeTrace(seed, changing, changing && index % 4 === 1, index % 7 === 0)),
129       y: Number(changing),
130     };
131   });
132 }
133
134 function probability(rawFeatures, model) {
135   let logit = model.weights.bias;
136   FEATURE_NAMES.forEach((name, index) => {
137     logit += model.weights[name] * ((rawFeatures[index] - model.means[name]) /
138   > model.stds[name]));
139   });
140   return sigmoid(logit);
141 }
142
143 export function trainSyntheticModel() {
144   // Separate fixed seeds make demo, fitting, and evaluation independent.
145   const training = makeDataset(320, 0x41d53a);
146   const heldOut = makeDataset(128, 0x91f337);
147   const means = {};
148   const stds = {};
149   FEATURE_NAMES.forEach((name, index) => {
150     means[name] = average(training.map((item) => item.x[index]));
151     stds[name] = Math.max(
152       Math.sqrt(average(training.map((item) => (item.x[index] - means[name]) ** 2))),
153       0.001,
154     );
155   });
156   const weights = Object.fromEntries(["bias", ...FEATURE_NAMES].map((name) => [name, 0]));
157   const model = { weights, means, stds };
158
159   // Batch logistic regression; all coefficients are inspectable in the UI.
160   for (let epoch = 0; epoch < 340; epoch += 1) {
161     const gradient = Object.fromEntries(["bias", ...FEATURE_NAMES].map((name) => [name, 0]));
162     for (const item of training) {
163       const error = probability(item.x, model) - item.y;
164       gradient.bias += error;
165       FEATURE_NAMES.forEach((name, index) => {
166         gradient[name] += error * ((item.x[index] - means[name]) / stds[name]);
167       });
168     }
169     const rate = 0.16;
170     weights.bias -= rate * gradient.bias / training.length;
171     FEATURE_NAMES.forEach((name) => {
172       weights[name] -= rate * (gradient[name] / training.length + 0.001 * weights[name]);
173     });
174   }
175
176   let tp = 0;
177   let tn = 0;
178   let fp = 0;
179   let fn = 0;
180   for (const item of heldOut) {
```

engine.js / continued

```

180     const predicted = probability(item.x, model) >= 0.5 ? 1 : 0;
181     if (predicted === 1 && item.y === 1) tp += 1;
182     else if (predicted === 0 && item.y === 0) tn += 1;
183     else if (predicted === 1) fp += 1;
184     else fn += 1;
185   }
186   model.metrics = {
187     accuracy: (tp + tn) / heldOut.length,
188     precision: tp + fp ? tp / (tp + fp) : 0,
189     recall: tp + fn ? tp / (tp + fn) : 0,
190     tp,
191     tn,
192     fp,
193     fn,
194   };
195   model.trainCount = training.length;
196   model.testCount = heldOut.length;
197   return model;
198 }
199
200 function inspectQuality(samples) {
201   const flags = [];
202   const badIndices = new Set();
203   if (!Array.isArray(samples) || samples.length !== 24) {
204     flags.push("Expected a complete 24-sample synthetic trace.");
205     return { valid: false, flags, badIndices };
206   }
207   samples.forEach((sample, index) => {
208     if (!sample || typeof sample !== "object") {
209       flags.push(`Sample ${index + 1}: missing reading set.`);
210       badIndices.add(index);
211       return;
212     }
213     if (!Number.isFinite(sample.t) || (index > 0 && (!samples[index - 1] || sample.t <=
214 > samples[index - 1].t))) {
215       flags.push(`Sample ${index + 1}: invalid time order.`);
216       badIndices.add(index);
217     }
218     for (const name of CHANNELS) {
219       const value = sample[name];
220       if (typeof value !== "number" || !Number.isFinite(value)) {
221         flags.push(`Sample ${index + 1}: ${name} is missing.`);
222         badIndices.add(index);
223       } else if (value < LIMITS[name][0] || value > LIMITS[name][1]) {
224         flags.push(`Sample ${index + 1}: ${name} is outside the simulated sensor range.`);
225         badIndices.add(index);
226       }
227     }
228   });
229   if (flags.length) return { valid: false, flags, badIndices };
230
231   const spikeLimit = { temperature: 1.1, moisture: 8, ph: 0.45, glucose: 8 };
232   for (const name of CHANNELS) {
233     for (let index = 1; index < samples.length - 1; index += 1) {
234       const before = samples[index - 1][name];
235       const value = samples[index][name];
236       const after = samples[index + 1][name];
237       if (Math.abs(value - (before + after) / 2) > spikeLimit[name] && Math.abs(before - after)
238 > < spikeLimit[name]) {
239         flags.push(`Sample ${index + 1}: isolated ${name} spike; recheck contact or
240 > calibration.`);
241         badIndices.add(index);
242       }
243     }
244   }
245   return { valid: true, flags, badIndices };
246 }

```

engine.js / continued

```

239     }
240   }
241   const lastIndex = samples.length - 1;
242   const previousStep = Math.abs(samples[lastIndex - 1][name] - samples[lastIndex - 2][name]);
243   const finalStep = Math.abs(samples[lastIndex][name] - samples[lastIndex - 1][name]);
244   if (finalStep > spikeLimit[name] * 2 && previousStep < spikeLimit[name]) {
245     flags.push(`Sample ${lastIndex + 1}: abrupt ${name} endpoint; recheck contact or
> calibration.`);
246     badIndices.add(lastIndex);
247   }
248 }
249 return { valid: flags.length === 0, flags, badIndices };
250 }
251
252 export function analyze(samples, model) {
253   const checked = inspectQuality(samples);
254   const last = Array.isArray(samples) && samples.length ? samples[samples.length - 1] : null;
255   const latest = Object.fromEntries(CHANNELS.map((name) => [name, last && typeof last ===
> "object" ? (last[name] ?? null) : null]));
256   const quality = { valid: checked.valid, flags: checked.flags };
257   if (!quality.valid) {
258     return {
259       quality,
260       patternScore: null,
261       decision: {
262         action: "recheck",
263         label: "Recheck simulated sensor",
264         rationale: "Data-quality gate held the synthetic score and all virtual treatment tokens
> at zero. Inspect the flagged readings.",
265         ultrasoundLevel: 0,
266         violetLevel: 0,
267       },
268       latest,
269       evidenceIndices: [...checked.badIndices].slice(0, 4),
270     };
271   }
272   if (!model || !model.weights || !model.means || !model.stds) {
273     throw new TypeError("A trained synthetic model is required.");
274   }
275   const patternScore = probability(features(samples), model);
276   const changing = patternScore >= 0.5;
277   const high = patternScore >= 0.88;
278   return {
279     quality,
280     patternScore,
281     decision: changing
282       ? {
283         action: "review",
284         label: "Prepare human review",
285         rationale: "The generated trace matches a synthetic changing-pattern class. The
> virtual levels illustrate a disconnected control policy, not a treatment recommendation.",
286         ultrasoundLevel: high ? 3 : 1,
287         violetLevel: high ? 2 : 1,
288       }
289       : {
290         action: "record",
291         label: "Record synthetic trace",
292         rationale: "The generated trace matches the stable-pattern class. Keep the
> measurements available for human review.",
293         ultrasoundLevel: 0,
294         violetLevel: 0,
295       },

```

engine.js / continued

```
296     latest,  
297     evidenceIndices: [0, 5, 18, 23],  
298   };  
299 }
```

tests/engine.test.mjs

```
1 import test from "node:test";
2 import assert from "node:assert/strict";
3 import { analyze, generateScenario, trainSyntheticModel } from "../engine.js";
4
5 const model = trainSyntheticModel();
6
7 test("demo traces are deterministic, complete, and use four bounded channels", () => {
8   for (const key of ["stable", "change", "fault"]) {
9     const scenario = generateScenario(key);
10    assert.deepEqual(scenario, generateScenario(key));
11    assert.equal(scenario.samples.length, 24);
12    assert.deepEqual(scenario.samples.map((sample) => sample.t), Array.from({ length: 24 }, (_,
13    > index) => index));
14    for (const sample of scenario.samples) {
15      assert.deepEqual(Object.keys(sample), ["t", "temperature", "moisture", "ph", "glucose"]);
16    }
17    assert.throws(() => generateScenario("infection"), RangeError);
18  });
19
20 test("classifier is fitted and evaluated on separate deterministic synthetic traces", () => {
21   assert.deepEqual(model, trainSyntheticModel());
22   assert.equal(model.trainCount, 320);
23   assert.equal(model.testCount, 128);
24   const { tp, tn, fp, fn, accuracy, precision, recall } = model.metrics;
25   assert.equal(tp + tn + fp + fn, model.testCount);
26   assert.equal(accuracy, (tp + tn) / model.testCount);
27   assert.equal(precision, tp / (tp + fp));
28   assert.equal(recall, tp / (tp + fn));
29   assert.ok(accuracy > 0.9);
30   assert.ok(accuracy < 1, "held-out synthetic examples should include ambiguous patterns");
31 });
32
33 test("stable and changing synthetic patterns lead to distinct bounded virtual actions", () => {
34   const stable = analyze(generateScenario("stable").samples, model);
35   const changing = analyze(generateScenario("change").samples, model);
36   assert.equal(stable.quality.valid, true);
37   assert.equal(changing.quality.valid, true);
38   assert.ok(stable.patternScore < 0.5);
39   assert.ok(changing.patternScore >= 0.5);
40   assert.equal(stable.decision.action, "record");
41   assert.equal(changing.decision.action, "review");
42   for (const result of [stable, changing]) {
43     assert.ok(result.evidenceIndices.every((index) => index >= 0 && index < 24));
44     assert.ok(result.decision.ultrasoundLevel >= 0 && result.decision.ultrasoundLevel <= 3);
45     assert.ok(result.decision.violetLevel >= 0 && result.decision.violetLevel <= 3);
46   }
47 });
48
49 test("missing and implausible readings fail closed even on a changing trace", () => {
50   const fault = analyze(generateScenario("fault").samples, model);
51   assert.equal(fault.quality.valid, false);
52   assert.equal(fault.patternScore, null);
53   assert.equal(fault.decision.action, "recheck");
54   assert.equal(fault.decision.ultrasoundLevel, 0);
55   assert.equal(fault.decision.violetLevel, 0);
56   assert.ok(fault.quality.flags.some((flag) => flag.includes("ph")));
57   assert.ok(fault.quality.flags.some((flag) => flag.includes("glucose")));
58   assert.deepEqual(fault.evidenceIndices, [21, 23]);
59 });
60
61 test("every channel, sample count, and isolated spike is quality-gated", () => {
```

tests/engine.test.mjs / continued

```
62   for (const channel of ["temperature", "moisture", "ph", "glucose"]) {
63     const samples = structuredClone(generateScenario("stable").samples);
64     samples[7][channel] = null;
65     const result = analyze(samples, model);
66     assert.equal(result.quality.valid, false);
67     assert.equal(result.patternScore, null);
68     assert.equal(result.decision.ultrasoundLevel, 0);
69   }
70   const short = generateScenario("stable").samples.slice(1);
71   assert.equal(analyze(short, model).quality.valid, false);
72   const noisy = structuredClone(generateScenario("stable").samples);
73   noisy[12].temperature += 2;
74   assert.equal(analyze(noisy, model).quality.valid, false);
75   const badEndpoint = structuredClone(generateScenario("stable").samples);
76   badEndpoint[23].temperature += 3;
77   assert.equal(analyze(badEndpoint, model).quality.valid, false);
78   };
```

submission/make_onepager.py

```
1  """Build the one-page Patchline project summary PDF."""
2
3  from pathlib import Path
4  from reportlab.pdfgen import canvas
5  from reportlab.lib.colors import HexColor
6
7
8  OUT = Path(__file__).with_name("Patchline-One-Page.pdf")
9  W, H = 612, 792
10 PAPER = HexColor("#F4F1EA")
11 INK = HexColor("#151A18")
12 MUTED = HexColor("#667670")
13 RULE = HexColor("#C9D0C9")
14 TERRA = HexColor("#CE5B3A")
15 PALE = HexColor("#E9E3D8")
16
17
18 def label(c, text, x, y, color=MUTED, size=8):
19     c.setFillColor(color)
20     c.setFont("Courier-Bold", size)
21     c.drawString(x, y, text.upper())
22
23
24 def body(c, lines, x, y, width=240, size=10.2, leading=14.5, color=INK):
25     c.setFillColor(color)
26     c.setFont("Helvetica", size)
27     for line in lines:
28         words = line.split()
29         current = ""
30         for word in words:
31             candidate = f"{current} {word}".strip()
32             if c.stringWidth(candidate, "Helvetica", size) > width and current:
33                 c.drawString(x, y, current)
34                 y -= leading
35                 current = word
36             else:
37                 current = candidate
38         if current:
39             c.drawString(x, y, current)
40             y -= leading
41         y -= 3
42     return y
43
44
45 def rule(c, y, x1=38, x2=574):
46     c.setStrokeColor(RULE)
47     c.setLineWidth(0.75)
48     c.line(x1, y, x2, y)
49
50
51 def rounded(c, x, y, w, h, fill, radius=6):
52     c.setFillColor(fill)
53     c.roundRect(x, y, w, h, radius, stroke=0, fill=1)
54
55
56 def main():
57     OUT.parent.mkdir(parents=True, exist_ok=True)
58     c = canvas.Canvas(str(OUT), pagesize=(W, H), pageCompression=1)
59     c.setTitle("Patchline | One-page project summary")
60     c.setAuthor("Rishik Rontala")
61     c.setFillColor(PAPER)
62     c.rect(0, 0, W, H, stroke=0, fill=1)
```

submission/make_onepager.py / continued

```
63
64     label(c, "PATCHLINE / SYSTEM STUDY 01", 38, 754, INK, 9)
65     label(c, "UNIVABIO 2026", 477, 754, TERRA, 8)
66     rule(c, 742)
67
68     c.setFillColor(INK)
69     c.setFont("Times-Roman", 29)
70     c.drawString(38, 704, "A closed-loop bandage,")
71     c.drawString(38, 673, "shown as a digital twin.")
72     rounded(c, 424, 671, 150, 40, INK, 4)
73     label(c, "SIMULATION ONLY", 440, 686, PAPER, 9)
74
75     body(c, [
76         "Patchline demonstrates how a future smart dressing could turn four wound-environment
77 > signals into a reviewable action - and stop when a sensor cannot be trusted."
78     ], 38, 645, 525, 11, 15)
79     rule(c, 596)
80
81     label(c, "01 SENSE", 38, 572, TERRA)
82     label(c, "02 LEARN", 181, 572, TERRA)
83     label(c, "03 ORCHESTRATE", 325, 572, TERRA)
84     label(c, "04 SHOW", 469, 572, TERRA)
85     c.setStrokeColor(RULE)
86     c.setLineWidth(1)
87     c.line(38, 557, 574, 557)
88     for x in [38, 181, 325, 469, 574]:
89         c.setFillColor(TERRA)
90         c.circle(x, 557, 2.7, stroke=0, fill=1)
91     body(c, ["pH, patch temperature, dressing wetness, wound-fluid glucose"], 38, 540, 118, 9.3,
92 > 12.5)
93     body(c, ["A small model trains and tests on separate synthetic traces"], 181, 540, 118, 9.3,
94 > 12.5)
95     body(c, ["A quality gate checks the score and may refuse to proceed"], 325, 540, 118, 9.3,
96 > 12.5)
97     body(c, ["Virtual ultrasound and violet states; evidence-linked handoff"], 469, 540, 105,
98 > 9.3, 12.5)
99
100     rule(c, 478)
101     label(c, "THE LIVE DEMO", 38, 453, INK, 8)
102     label(c, "WHAT THE PROTOTYPE PROVES", 314, 453, INK, 8)
103
104     c.setFillColor(INK)
105     c.setFont("Times-Roman", 18)
106     c.drawString(38, 428, "From change to restraint")
107     c.drawString(314, 428, "A visible decision loop")
108
109     body(c, [
110         "1 Select a stable stream. The virtual patch records four generated channels.",
111 > "2 Select a changing stream. The trained model scores a synthetic pattern; the
112 > orchestrator explains a virtual response.",
113 > "3 Inject a pH fault. The quality gate cancels virtual outputs and asks for a recheck
114 > and human review."
115     ], 38, 407, 235, 9.7, 13.2)
116
117     body(c, [
118         "Reproducible sensor streams, manual controls, held-out synthetic evaluation, event
119 > history, and a simulated board/firmware view.",
120 > "Every decision shows its input window and reason. Virtual outputs use abstract levels
121 > 0-3; they are not physical settings."
122     ], 314, 407, 252, 9.7, 13.2)
123
124     rounded(c, 38, 191, 536, 83, PALE, 5)
```

submission/make_onepager.py / continued

```
116     c.setFillColor(TERRA)
117     c.rect(38, 191, 4, 83, stroke=0, fill=1)
118     label(c, "RESEARCH BOUNDARY", 54, 253, TERRA, 8)
119     body(c, [
120         "No patient data, physical patch, clinical validation, infection diagnosis, or delivered
121         > treatment. Synthetic labels test software behavior only. Wound-fluid glucose is not blood
122         > glucose. This concept cannot guide care."
123         ], 54, 235, 499, 9.6, 13.1)
124     label(c, "EVIDENCE CONTEXT", 38, 164, INK, 8)
125     body(c, [
126         "IWGDF/IDSA: diabetes-related foot infection requires clinical assessment; foot
127         > temperature alone is not a diagnostic test. IWGDF wound-healing guidance does not establish this
128         > proposed physical-therapy approach as routine care."
129         ], 38, 147, 536, 8.5, 11.3, MUTED)
130     c.setFillColor(MUTED)
131     c.setFont("Helvetica", 7.2)
132     c.drawString(38, 88, "Sources: idsociety.org/practice-guideline/diabetic-foot-infections/")
133     c.drawString(38, 78, "onlinelibrary.wiley.com/doi/10.1002/dmrr.3644")
134     rule(c, 61)
135     label(c, "BUILT BY RISHIK RONTALA", 38, 41, INK, 8)
136     label(c, "INTERACTIVE PROTOTYPE + OPEN CODE", 349, 41, MUTED, 7)
137     c.showPage()
138     c.save()
139     print(OUT)
140 if __name__ == "__main__":
141     main()
```

submission/record-demo.cjs

```
1  'use strict';
2
3  // Records the real local Patchline prototype. No audio, network API, or patient data.
4  const { chromium } = require('playwright');
5  const fs = require('fs');
6  const path = require('path');
7
8  const url = process.env.PATCHLINE_URL || 'http://127.0.0.1:8765/';
9  const rehearsal = process.argv.includes('--rehearse');
10 const videoPath = path.join(__dirname, 'Patchline-Demo.webm');
11
12 const pause = (page, ms) => page.waitForTimeout(ms);
13
14 async function visible(page, selector, name) {
15   const control = page.locator(selector).first();
16   if (!await control.isVisible()) throw new Error(`Missing ${name}: ${selector}`);
17   console.log(`OK ${name}`);
18   return control;
19 }
20
21 async function check(page, selector, expected, name) {
22   const actual = (await page.locator(selector).first().textContent())?.trim() || '';
23   if (!actual.includes(expected)) throw new Error(`${name}: expected ${expected}, saw
24 > ${actual}`);
25   console.log(`OK ${name}: ${actual.slice(0, 80)}`);
26 }
27
28 async function injectOverlays(page) {
29   await page.evaluate(() => {
30     const cursor = document.createElement('div');
31     cursor.id = 'demo-cursor';
32     cursor.style.cssText = 'position:fixed;left:0;top:0;z-index:999999;pointer-events:none;width
33 > :28px;height:28px;filter:drop-shadow(2px 3px 3px rgba(0,0,0,.3));transition:left .10s ease,top
34 > .10s ease';
35     cursor.innerHTML = '<svg width="28" height="28" viewBox="0 0 28 28" fill="none"
36 > xmlns="http://www.w3.org/2000/svg"><path d="M4 2L23 15L14 16L10 25L4 22" fill="#F4F1EA"
37 > stroke="#100F0D" stroke-width="2" stroke-linejoin="round"/></svg>';
38     document.body.append(cursor);
39     document.addEventListener('mousemove', event => {
40       cursor.style.left = `${event.clientX}px`;
41       cursor.style.top = `${event.clientY}px`;
42     });
43     const caption = document.createElement('div');
44     caption.id = 'demo-caption';
45     caption.style.cssText = 'position:fixed;left:24px;right:24px;bottom:18px;z-index:999998;po
46 > inter-events:none;background:rgba(16,15,13,.94);color:#F4F1EA;font:600 19px/1.35
47 > Arial,sans-serif;letter-spacing:.01em;padding:13px 20px;border-left:5px solid
48 > #DA532C;max-width:1160px;box-sizing:border-box';
49     caption.textContent = 'SIMULATED PROTOTYPE · No physical patch or treatment';
50     document.body.append(caption);
51   });
52 }
53
54 async function caption(page, text) {
55   await page.locator('#demo-caption').evaluate((el, value) => { el.textContent = value; },
56 > text);
57   await pause(page, 350);
58 }
59
60 async function moveTo(page, control) {
61   await control.scrollIntoViewIfNeeded();
62   await pause(page, 350);
63 }
```

submission/record-demo.cjs / continued

```
54   const box = await control.boundingBox();
55   if (!box) throw new Error('Target has no bounding box');
56   await page.mouse.move(box.x + box.width / 2, box.y + box.height / 2, { steps: 14 });
57   await pause(page, 300);
58 }
59
60 async function click(page, selector, name) {
61   const control = await visible(page, selector, name);
62   await moveTo(page, control);
63   await control.click();
64   await pause(page, 700);
65 }
66
67 async function scrollTo(page, selector, name) {
68   const target = await visible(page, selector, name);
69   await target.evaluate(el => el.scrollIntoView({ behavior: 'smooth', block: 'start' }));
70   await pause(page, 1300);
71 }
72
73 (async () => {
74   const browser = await chromium.launch({
75     channel: 'chrome', headless: true,
76     args: ['--no-sandbox', '--disable-gpu', '--mute-audio']
77   });
78   const context = await browser.newContext({
79     viewport: { width: 1280, height: 720 },
80     ...(rehearsal ? {} : { recordVideo: { dir: path.join(__dirname, '.video-tmp'), size: {
81 > width: 1280, height: 720 } } })
81   });
82   const page = await context.newPage();
83   const errors = [];
84   page.on('pageerror', error => errors.push(error.message));
85   await page.goto(url, { waitUntil: 'networkidle' });
86
87   try {
88     if (rehearsal) {
89       const fieldMap = await page.evaluate(() =>
90 > Array.from(document.querySelectorAll('button,a,input'))
91 > .filter(el => el.getBoundingClientRect().width > 0)
92 > .map(el => ({ tag: el.tagName, id: el.id, text: el.textContent.trim().slice(0, 40),
93 > scenario: el.dataset.scenario || '' })));
94       console.log('FIELD MAP', JSON.stringify(fieldMap));
95       await visible(page, 'a[href="#prototype"]', 'prototype link');
96       await visible(page, '[data-scenario="stable"]', 'stable control');
97       await visible(page, '[data-scenario="change"]', 'change control');
98       await visible(page, '[data-scenario="fault"]', 'fault control');
99       await visible(page, '#trace-grid', 'sensor traces');
100      await visible(page, '#decision-label', 'bounded decision');
101      await visible(page, '#handoff', 'handoff');
102      await visible(page, '#print-report', 'print report control');
103      await visible(page, '#boundaries', 'boundaries');
104      await page.locator('[data-scenario="stable"]').click();
105      await check(page, '#decision-label', 'Record', 'stable decision');
106      await page.locator('[data-scenario="change"]').click();
107      await check(page, '#decision-label', 'review', 'change decision');
108      await page.locator('[data-scenario="fault"]').click();
109      await check(page, '#score-value', '-', 'fault withheld score');
110      await check(page, '#ultrasound-value', '0 / 3', 'fault ultrasound zero');
111      await check(page, '#violet-value', '0 / 3', 'fault violet zero');
112      await visible(page, '#claim-list button', 'evidence link');
113      if (errors.length) throw new Error('Page errors: ${errors.join('; ')}');
114      console.log('REHEARSAL PASSED');
```

submission/record-demo.cjs / continued

```
113     } else {
114       await injectOverlays(page);
115       await page.mouse.move(1170, 80, { steps: 10 });
116       await caption(page, 'A smart bandage concept you can question');
117       await pause(page, 3300);
118       await click(page, 'a[href="#prototype"]', 'Explore prototype');
119       await caption(page, 'Four generated sensor channels. No patient readings.');
```

120 await pause(page, 1500);

121 await click(page, '[data-scenario="stable"]', 'Stable trace');

122 await scrollTo(page, '.trace-panel', 'sensor traces');

123 await pause(page, 2800);

124 await scrollTo(page, '.decision-panel', 'control plane');

125 await caption(page, 'Stable trace: record only. Both virtual outputs stay at zero.');

126 await pause(page, 2700);

127 await scrollTo(page, '.tuning-panel', 'last-sample controls');

128 await caption(page, 'You can change the last synthetic reading yourself.');

129 await click(page, '#manual-temperature', 'temperature slider');

130 for (let index = 0; index < 4; index += 1) {

131 await page.locator('#manual-temperature').press('ArrowRight');

132 await pause(page, 300);

133 }

134 await pause(page, 2500);

135 await click(page, '#reset-controls', 'Reset last sample');

136 await pause(page, 1000);

137

138 await scrollTo(page, '.scenario-bar', 'scenario controls');

139 await click(page, '[data-scenario="change"]', 'Pattern change trace');

140 await caption(page, 'Changing pattern: prepare a human review.');

141 await scrollTo(page, '.decision-panel', 'changed decision');

142 await pause(page, 3400);

143 await caption(page, '0-3 indicators are abstract tokens, never doses.');

144 await pause(page, 2700);

145

146 await scrollTo(page, '.scenario-bar', 'scenario controls');

147 await click(page, '[data-scenario="fault"]', 'Sensor fault trace');

148 await caption(page, 'Bad evidence stops the loop before the model can act.');

149 await scrollTo(page, '.trace-panel', 'fault traces');

150 await pause(page, 2400);

151 await scrollTo(page, '.decision-panel', 'quality hold decision');

152 await pause(page, 3300);

153 await caption(page, 'Score withheld. Virtual ultrasound and violet light: zero.');

154 await pause(page, 2600);

155

156 await scrollTo(page, '#handoff', 'reviewable handoff');

157 await caption(page, 'Every claim links to the source synthetic samples.');

158 await pause(page, 2300);

159 await click(page, '#claim-list .claim-row:last-child button', 'View flagged samples');

160 await pause(page, 2600);

161 await scrollTo(page, '#handoff', 'handoff and model provenance');

162 await caption(page, 'Model provenance and firmware-like events stay visible.');

163 await pause(page, 3600);

164 await caption(page, 'A one-page report can be printed for human review.');

165 await page.emulateMedia({ media: 'print' });

166 await pause(page, 4800);

167 await page.emulateMedia({ media: 'screen' });

168 await pause(page, 1000);

169

170 await scrollTo(page, '#boundaries', 'prototype boundaries');

171 await caption(page, 'No physical sensors. No infection diagnosis. No treatment.');

172 await pause(page, 4300);

173 await caption(page, 'PATCHLINE · Built by Rishik Rontala');

174 await pause(page, 3500);

submission/record-demo.cjs / continued

```
175         if (errors.length) throw new Error(`Page errors: ${errors.join('; ')}`);
176         console.log('RECORDING FLOW PASSED');
177     }
178 } finally {
179     await context.close();
180     if (!rehearsal) {
181         await page.video().saveAs(videoPath);
182         console.log(`VIDEO ${videoPath} ${fs.statSync(videoPath).size} bytes`);
183     }
184     await browser.close();
185 }
186 })().catch(error => { console.error(error.stack || error.message); process.exitCode = 1; });
```

submission/make_code_pdf.py

```
1  """Build a readable PDF archive of all Patchline source code for Devpost."""
2
3  from pathlib import Path
4  import textwrap
5
6  from reportlab.lib.colors import HexColor
7  from reportlab.pdfbase import pdfmetrics
8  from reportlab.pdfbase.ttfonts import TTFont
9  from reportlab.pdfgen import canvas
10
11
12  ROOT = Path(__file__).resolve().parents[1]
13  OUTPUT = ROOT / "submission" / "Patchline-Code.pdf"
14  SOURCES = [
15      "index.html",
16      "demo.html",
17      "styles.css",
18      "app.js",
19      "engine.js",
20      "tests/engine.test.mjs",
21      "submission/make_onepager.py",
22      "submission/record-demo.cjs",
23      "submission/make_code_pdf.py",
24  ]
25
26  PAGE_W, PAGE_H = 612, 792
27  LEFT, RIGHT, TOP, BOTTOM = 43, 43, 48, 45
28  LEADING = 10.6
29  FONT_SIZE = 7.7
30  INK = HexColor("#100F0D")
31  TERRA = HexColor("#C24C2B")
32  FAINT = HexColor("#BEB9AF")
33  PAPER = HexColor("#F8F6F0")
34
35  pdfmetrics.registerFont(TTFont("Andale", "/System/Library/Fonts/Supplemental/Andale Mono.ttf"))
36  pdfmetrics.registerFont(TTFont("Arial", "/Library/Fonts/Arial Unicode.ttf"))
37
38
39  def base_page(pdf, page_number, label):
40      pdf.setFillColor(PAPER)
41      pdf.rect(0, 0, PAGE_W, PAGE_H, fill=1, stroke=0)
42      pdf.setStrokeColor(FAINT)
43      pdf.setLineWidth(0.5)
44      pdf.line(LEFT, PAGE_H - 37, PAGE_W - RIGHT, PAGE_H - 37)
45      pdf.line(LEFT, BOTTOM - 10, PAGE_W - RIGHT, BOTTOM - 10)
46      pdf.setFont("Andale", 7.5)
47      pdf.setFillColor(TERRA)
48      pdf.drawString(LEFT, PAGE_H - 27, "PATCHLINE / SOURCE ARCHIVE")
49      pdf.setFillColor(INK)
50      pdf.drawRightString(PAGE_W - RIGHT, PAGE_H - 27, label.upper()[:50])
51      pdf.setFont("Andale", 7)
52      pdf.drawString(LEFT, BOTTOM - 24, "SIMULATED PROTOTYPE · NO MEDICAL USE")
53      pdf.drawRightString(PAGE_W - RIGHT, BOTTOM - 24, f"{{page_number:02d}}")
54
55
56  pdf = canvas.Canvas(str(OUTPUT), pagesize=(PAGE_W, PAGE_H), pageCompression=1)
57  pdf.setTitle("Patchline – Complete Source Code")
58  pdf.setAuthor("Rishik Rontala")
59  page_number = 1
60  base_page(pdf, page_number, "INDEX")
61  pdf.setFillColor(INK)
62  pdf.setFont("Arial", 34)
```

submission/make_code_pdf.py / continued

```
63 pdf.drawString(LEFT, PAGE_H - 118, "Patchline")
64 pdf.setFont("Arial", 20)
65 pdf.drawString(LEFT, PAGE_H - 148, "Complete source code")
66 pdf.setFont("Arial", 10)
67 pdf.drawString(LEFT, PAGE_H - 184, "Built by Rishik Rontala · 6 October 2026")
68 pdf.drawString(LEFT, PAGE_H - 201, "Public repository: github.com/rishikrrontala-bot/patchline")
69 pdf.drawString(LEFT, PAGE_H - 228, "This archive includes every source file for the browser
> prototype, tests,")
70 pdf.drawString(LEFT, PAGE_H - 244, "demo recorder, and PDF generators. Generated media and PDFs
> are excluded.")
71 y = PAGE_H - 290
72 pdf.setFont("Andale", 9)
73 for index, source in enumerate(SOURCES, 1):
74     pdf.setFillColor(TERRA)
75     pdf.drawString(LEFT, y, f"{index:02d}")
76     pdf.setFillColor(INK)
77     pdf.drawString(LEFT + 35, y, source)
78     y -= 24
79 pdf.setFont("Arial", 10)
80 pdf.setFillColor(INK)
81 pdf.drawString(LEFT, BOTTOM + 55, "All wound readings and labels in the prototype are
> synthetic.")
82 pdf.drawString(LEFT, BOTTOM + 38, "The code cannot diagnose infection or operate treatment
> hardware.")
83 pdf.showPage()
84
85 for source in SOURCES:
86     lines = (ROOT / source).read_text(encoding="utf-8").splitlines()
87     page_number += 1
88     base_page(pdf, page_number, source)
89     y = PAGE_H - TOP - 16
90     pdf.setFillColor(INK)
91     pdf.setFont("Arial", 15)
92     pdf.drawString(LEFT, y, source)
93     y -= 24
94     for line_number, line in enumerate(lines, 1):
95         # Monospaced code wraps at the printable width; continuation rows are marked.
96         chunks = textwrap.wrap(
97             line.expandtabs(2), width=96, break_long_words=True,
98             break_on_hyphens=False, replace_whitespace=False, drop_whitespace=False,
99             ) or [""]
100     for chunk_index, chunk in enumerate(chunks):
101         if y < BOTTOM + 2:
102             pdf.showPage()
103             page_number += 1
104             base_page(pdf, page_number, source)
105             y = PAGE_H - TOP - 16
106             pdf.setFont("Arial", 15)
107             pdf.setFillColor(INK)
108             pdf.drawString(LEFT, y, f"{source} / continued")
109             y -= 24
110             pdf.setFont("Andale", FONT_SIZE)
111             pdf.setFillColor(TERRA)
112             pdf.drawRightString(LEFT + 30, y, str(line_number) if chunk_index == 0 else ">")
113             pdf.setFillColor(INK)
114             pdf.drawString(LEFT + 41, y, chunk)
115             y -= LEADING
116 pdf.showPage()
117
118 pdf.save()
119 print(OUTPUT)
```